Labsheet 5 Fullstack Lab

Vishavjeet Baraily
cu24250187

Q1. Create a new folder named express-lab and initialize it with npm init -y.

Ans. Open the terminal, create the folder, move into it, and initialize the Node.js project.

```
mkdir express-lab
cd express-lab npm
init -y
```

Q2. Install express, and install nodemon as a dev dependency.

Ans. Install Express normally and nodemon as a development dependency.

```
npm install express npm
install --save-dev nodemon
```

Q3. Create index.js with a basic server on port 4000 that responds with "Express Lab Running" on GET /.

Ans. Create an Express server and listen on port 4000.

```
const express = require("express");
const app = express();

app.get("/", (req, res) => {
res.send("Express Lab Running"); });

app.listen(4000, () => {
console.log("Server running on port 4000"); });
```

Q4. Add a "dev" script in package.json that runs the server using nodemon.

Ans. Add the following script inside the scripts object.

```
"scripts": {
  "start": "node index.js",
  "dev": "nodemon index.js"
}
```

Q5. Create GET /about that returns your name and roll number as a JSON object.

Ans. The route returns the student name and CUID as JSON.

```
app.get("/about", (req, res) => {
res.json({     name: "Nilay Vaishnava",
rollNumber:
"CU24250085"   });
});
```

Q6. Create GET /courses that returns an array of any 3 course names as JSON.

Ans. The route returns an array containing three course names.

```
app.get("/courses", (req, res) => {   res.json(["Data Structures",
"DBMS", "Full Stack Web Development"]); });
```

Q7. Create POST /echo that reads the JSON body sent by the client and returns the exact same data back.

Ans. Enable JSON parsing and return req.body directly. `app.use(express.json());`

```
app.post("/echo", (req, res) => {
res.json(req.body); });
```

Q8. Create GET /students/:id that returns { id, message: "Student details" } using the id from the URL.

Ans. Read the route parameter from req.params.id.

```
app.get("/students/:id", (req, res) => {
res.json({     id: req.params.id,     message:
"Student details"   });
});
```

Q9. Create GET /search that reads query parameters name and age, and returns them in the JSON response.

Ans. Query parameters are available through req.query.

```
app.get("/search", (req, res) => {
const { name, age } = req.query;   res.json({
name, age }); });
```

Q10. Create GET /products/:category/:id that returns both the category and id captured from the URL.

Ans. Read both dynamic route parameters from req.params.

```
app.get("/products/:category/:id", (req, res) => {
const { category, id } = req.params;   res.json({ category,
id }); });
```

Q11. Write a custom logger middleware that prints the request method, URL, and a timestamp for every incoming request. Apply it globally with app.use().

Ans. The middleware prints the request information and then calls next() so the request continues.

```
function   logger(req,   res,   next)   {    console.log(`${new
Date().toISOString()} ${req.method} ${req.url}`);  next(); }
app.use(logger);
```

Q12. Write a middleware that checks for a custom header x-api-key. If it is missing, respond with 403 Forbidden. Apply this middleware only to routes starting with /admin.

Ans. The middleware is mounted on /admin, so it affects only admin routes.

```
function checkApiKey(req, res, next) {
const apiKey = req.header("x-api-key");

  if (!apiKey) {     return res.status(403).json({
error: "Forbidden" });
}

  next(); } app.use("/admin",
checkApiKey);
```

Q13. Create a protected route GET /admin/dashboard that only returns data when the x-api-key header is present.

Ans. Because checkApiKey is mounted on /admin, the route is protected automatically.

```
app.get("/admin/dashboard", (req, res) => {
res.json({     message: "Welcome to admin
dashboard",     data: "Protected admin
data"   }); });
```

Q14. Enable express.json() and express.urlencoded({ extended: true }) as middleware.

Ans. Add both body-parsing middleware functions before routes that need request bodies.

```
app.use(express.json());
app.use(express.urlencoded({ extended: true }));
```

Q15. Create POST /register that accepts { name, email, password } in the request body and returns a success message including the submitted name.

Ans. Read the submitted fields from req.body and return a success response.

```
app.post("/register", (req, res) => {    const
{ name, email, password } = req.body;

  res.json({     message: `Registration successful
for ${name}`   });
});
```

Q16. Create POST /contact that accepts a urlencoded form submission, logs it to the console, and returns a thank-you message.

Ans. express.urlencoded() parses the form and the route logs the submitted object.

```
app.post("/contact", (req, res) => {
console.log(req.body);    res.send("Thank you
for contacting us"); });
```

Q17. Create a routes folder. Move all /students routes from Part C into routes/students.js using express.Router().

Ans. Create a router file and export it. The router contains the student routes.

```
// routes/students.js const express
= require("express"); const router
= express.Router();

router.get("/:id", (req, res) => {
res.json({     id: req.params.id,     message:
"Student details"   });
}); module.exports =

router;
```

Q18. Mount the students router inside index.js under the path /api/students.

Ans. Import the router and mount it on /api/students. The complete endpoint becomes /api/students/:id.

```
const studentsRouter = require("./routes/students"); app.use("/api/students", studentsRouter);
```

Q19. Repeat the same pattern for a new resource, books: GET /api/books (all books), GET /api/books/:id (one book), POST /api/books (add a book).

Ans. Use a separate router for books and keep the data in an in-memory array.

```
// routes/books.js const express =
require("express"); const router = express.Router();

let books = [   { id: 1, title: "JavaScript Basics", author:
"John Doe" },   { id: 2, title: "Node.js Guide", author:
"Jane Doe" } ]; router.get("/", (req, res) =>

res.json(books));

router.get("/:id", (req, res) => {   const book = books.find(b =>
b.id === Number(req.params.id));   if (!book) return
res.status(404).json({ error: "Book not found" });   res.json(book);
});

router.post("/", (req, res) => {   const book = { id:
books.length + 1, ...req.body };   books.push(book);
res.status(201).json(book);

}); module.exports =

router;

// index.js const booksRouter =
require("./routes/books"); app.use("/api/books",
booksRouter);
```

Q20. Choose any resource of your own and build all 5 CRUD routes for it: GET /api/, GET /api//:id, POST /api/, PUT /api//:id, DELETE /api//:id.

Ans. The resource chosen is tasks. The array stores tasks in memory and each route performs one CRUD operation.

```
let tasks = [   { id: 1, title: "Complete Express Lab",
completed: false } ];
app.get("/api/tasks", (req, res) => {
res.json(tasks); });

app.get("/api/tasks/:id", (req, res) => {    const task = tasks.find(t
=> t.id === Number(req.params.id));   if (!task) return
res.status(404).json({ error: "Task not found" });   res.json(task);
});

app.post("/api/tasks", (req, res) => {   const task = {
id: tasks.length ? tasks[tasks.length - 1].id + 1 : 1,
...req.body   };   tasks.push(task);   res.status(201).json(task);
});

app.put("/api/tasks/:id", (req, res) => {   const index = tasks.findIndex(t =>
t.id   ===   Number(req.params.id));        if   (index   ===   -1)   return
res.status(404).json({ error: "Task not found" });   tasks[index] = { id:
tasks[index].id, ...req.body };   res.json(tasks[index]); });

app.delete("/api/tasks/:id", (req, res) => {   const index =
tasks.findIndex(t => t.id === Number(req.params.id));   if (index === -1)
return res.status(404).json({ error: "Task not found" });    const deleted
= tasks.splice(index, 1)[0];   res.json(deleted); });
```

Q21. Test all 5 routes using Postman or Thunder Client. Take a screenshot of each successful response for your submission.

Ans. Send requests to the five task endpoints. A successful test should return HTTP 200 for GET, PUT and DELETE, and HTTP 201 for POST. Example test URLs are http://localhost:4000/api/tasks and http://localhost:4000/api/tasks/1. Screenshots are normally taken from Postman or Thunder Client after each successful response.

Q22. Add a 404 handler after all routes that returns { error: "Route not found" } for any unmatched URL.

Ans. Place the handler after every valid route.

```
app.use((req, res) => {   res.status(404).json({
error: "Route not found" }); });
```

Q23. Add a centralized error-handling middleware with four parameters that logs the error and sends back a JSON error response with the correct status code.

Ans. The middleware must have the signature (err, req, res, next) and should be registered after routes and the 404 handler.

```
app.use((err, req, res, next) => {
console.error(err);   const status =
err.statusCode || 500; res.status(status).json({
error: err.message
|| "Internal Server Error"   });
});
```

Q24. In the CRUD API from Part G, make GET /api//:id call next(err) with a 404 error when the given id does not exist.

Ans. Create an Error object, assign statusCode 404, and pass it to next(err).

```
app.get("/api/tasks/:id", (req, res, next) => {   const task =
tasks.find(t => t.id === Number(req.params.id));

  if (!task) {     const err = new
Error("Task not found");     err.statusCode
= 404;     return next(err);   }

  res.json(task);
});
```

```
Server running on port 4000
GET /
Express Lab Running

GET /about
{
  "name": "Nilay Vaishnava",
  "rollNumber": "CU24250085"
}

GET /courses
[
  "Data Structures",
  "DBMS",
  "Full Stack Web Development"
]
```

```
GET /students/101
{
  "id": "101",
  "message": "Student details"
}

GET /search?name=Yash&age=21
{
  "name": "Yash",
  "age": "21"
}

GET /products/books/101
{
  "category": "books",
  "id": "101"
}
```

```
GET /admin/dashboard
{
  "message": "Welcome to admin dashboard",
  "data": "Protected admin data"
}

GET /api/books
[
  {
    "id": 1,
    "title": "JavaScript Basics",
    "author": "John Doe"
  },
  {
    "id": 2,
    "title": "Node.js Guide",
    "author": "Jane Doe"
  }
]
```

```
GET /api/books/1
{
  "id": 1,
  "title": "JavaScript Basics",
  "author": "John Doe"
}

POST /api/books
{
  "id": 3,
  "title": "Express.js",
  "author": "Yash"
}

GET /api/tasks
[
  {
    "id": 1,
    "title": "Complete Express Lab",
    "completed": false
  }
]
```